

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA0401

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:25

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct:

I G. Yashmita (192511032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Yashmita

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

S.No.	Question	Mark
1	A server with a total storage capacity of 50 GB needs to effectively manage space for rapidly growing system logs, user files, and application data. Analyze the scenario and design an optimized Linux file system layout, incorporating suitable partitioning techniques and storage management tools (such as log rotation), to ensure efficient utilization of disk space and maintain system stability. The solution must ensure that log files do not exceed 10 GB and that critical services continue to operate without interruption due to disk space exhaustion.	6
2	A Linux system handling high CPU workloads suffers from frequent context switching delays. Given that hardware upgrades are not possible and real-time processing must be supported, evaluate kernel-level optimization techniques such as scheduler tuning and the use of kernel modules—to improve overall system performance.	5
3	An administrator must process 1 million log entries per day using only shell scripting. Analyze the problem and design an efficient solution using optimized Linux commands that ensures the script completes within 2 minutes. Justify the choice of	5

	commands used for achieving high performance	
4	A Linux system runs multiple background services, where a faulty process is consuming 90% of CPU resources. Without restarting the system and while ensuring critical services remain unaffected, analyze the situation and propose a process management strategy using appropriate Linux tools to identify, monitor, and control the faulty process.	5
5	A physical server must host 5 virtual machines with a total available RAM of 16 GB. Each VM requires a minimum of 2 GB, while the host operating system needs 4 GB. Analyze the scenario and propose an efficient resource allocation strategy using virtualization concepts to ensure all virtual machines operate smoothly under the given constraints.	4

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Optimized Linux File System Layout – 50 GB

Analysis

The server has a total storage capacity of 50 GB and must store:

- System files
- Rapidly growing system logs
- User files
- Application data

The major constraint is:

Log files must never exceed 10 GB.

A suitable partition layout is:

Partition	Size	Purpose
/boot	1 GB	Boot files
/	10 GB	Operating system
/var/log	10 GB	System/application logs
/home	10 GB	User files
/opt	12 GB	Application data
Swap	4 GB	Virtual memory
Free/Reserved	3 GB	Emergency growth
Total	50 GB	

Storage management strategy

1. Place /var/log in a separate 10 GB partition.
2. Use logrotate to rotate logs regularly.
3. Compress old logs.
4. Keep a fixed number of rotated logs.
5. Use monitoring tools such as df and du.
6. Reserve 3 GB for emergency situations.
7. Prevent /var/log from consuming space needed by critical services.

Example logrotate policy:

```
/var/log/*.log {  
    daily  
    rotate 7  
    compress
```

missingok
notifempty

}

This ensures logs are periodically rotated and compressed.

C Program

```
#include <stdio.h>
int main() {
int total = 50;
int boot = 1;
int root = 10;
int logs = 10;
int home = 10;
int app = 12;
int swap = 4;
int reserved = 3;
int used = boot + root + logs + home + app + swap + reserved;
printf("Linux File System Layout\n");
printf("-----\n");
printf("/boot : %d GB\n", boot);
printf("/ : %d GB\n", root);
printf("/var/log : %d GB\n", logs);
printf("/home : %d GB\n", home);
printf("/opt : %d GB\n", app);
printf("Swap : %d GB\n", swap);
printf("Reserved : %d GB\n", reserved);
printf("\nTotal Storage : %d GB\n", total);
printf("Allocated : %d GB\n", used);
if (logs <= 10)
printf("\nConstraint satisfied: Logs <= 10 GB\n");
else
printf("\nConstraint violated: Logs exceed 10 GB\n");
printf("Log management: logrotate + compression\n");
printf("Monitoring: df and du\n");
return 0;
}
```

OUTPUT :

```
Output

Linux File System Layout
-----
/boot      : 1 GB
/          : 10 GB
/var/log   : 10 GB
/home      : 10 GB
/opt       : 12 GB
Swap       : 4 GB
Reserved   : 3 GB

Total Storage : 50 GB
Allocated     : 50 GB

Constraint satisfied: Logs <= 10 GB
Log management: logrotate + compression
Monitoring: df and du
```

Conclusion: A separate 10 GB /var/log partition combined with logrotate, compression and reserved space prevents logs from exhausting the entire disk and protects critical services.

2. Kernel-Level Optimization for High CPU Workloads

Analysis

The system experiences frequent context switching, which causes CPU overhead and delays.

Constraints

- Hardware upgrade is not possible.
- Real-time processing is required.
- CPU performance must be improved.
- Critical tasks should receive higher priority.

Suitable techniques

1. Scheduler tuning

- o Give important processes higher priority.
- o Use nice and renice.
- o Use real-time scheduling policies such as SCHED_FIFO or SCHED_RR where appropriate.

2. CPU affinity

- o Use taskset to bind processes to selected CPUs.
- o This can reduce unnecessary CPU migration.

3. Kernel modules

- o Load only required modules.
- o Avoid unnecessary modules consuming resources.
- o Kernel modules can provide optimized functionality without rebuilding the entire kernel.

4. Reduce unnecessary processes

- o Disable services that are not required.

5. Real-time scheduling

- o Critical processing can be assigned a higher scheduling priority.

C Program

This program simulates selecting a scheduling strategy based on workload and context-switching level.

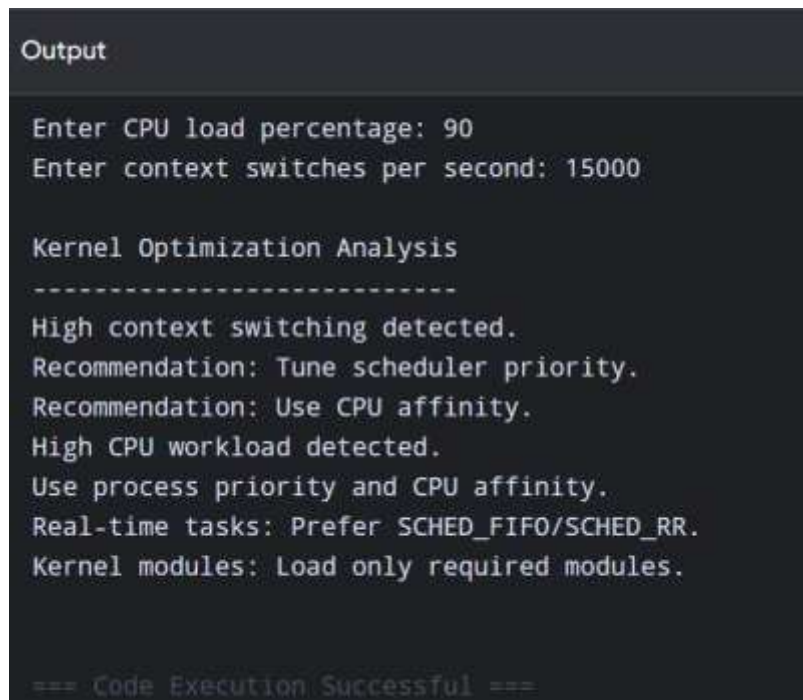
```
#include <stdio.h>
int main() {
    int cpu_load, context_switches;
    printf("Enter CPU load percentage: ");
    scanf("%d", &cpu_load);
    printf("Enter context switches per second: ");
    scanf("%d", &context_switches);
    printf("\nKernel Optimization Analysis\n");
    printf("-----\n");
    if (context_switches > 10000) {
        printf("High context switching detected.\n");
        printf("Recommendation: Tune scheduler priority.\n");
        printf("Recommendation: Use CPU affinity.\n");
    } else {
        printf("Context switching is within acceptable range.\n");
    }
}
```

```

if (cpu_load > 80) {
printf("High CPU workload detected.\n");
printf("Use process priority and CPU affinity.\n");
} else {
printf("CPU workload is acceptable.\n");
}
printf("Real-time tasks: Prefer SCHED_FIFO/SCHED_RR.\n");
printf("Kernel modules: Load only required modules.\n");
return 0;
}

```

OUTPUT :



```

Output

Enter CPU load percentage: 90
Enter context switches per second: 15000

Kernel Optimization Analysis
-----
High context switching detected.
Recommendation: Tune scheduler priority.
Recommendation: Use CPU affinity.
High CPU workload detected.
Use process priority and CPU affinity.
Real-time tasks: Prefer SCHED_FIFO/SCHED_RR.
Kernel modules: Load only required modules.

=== Code Execution Successful ===

```

Conclusion: Scheduler tuning, CPU affinity, process priority and selective kernel-module usage can reduce scheduling overhead without requiring hardware upgrades.

3. Processing 1 Million Log Entries Using Shell Commands

Analysis

The administrator must process 1 million log entries per day using only shell scripting.

Constraint

The entire operation must complete within 2 minutes. For high performance, commands that process data in a streaming manner should be used.

Recommended commands

- awk – fast filtering and counting
- grep – efficient pattern matching
- sort – sorting when required
- uniq – counting repeated values
- cut – extracting fields

Instead of processing each line using a shell for loop, use tools such as awk, because repeatedly starting shell commands creates significant overhead. For example:

```
awk '{count[$3]++} END {for (x in count) print x, count[x]}' logfile
```

C Program

This simulates processing 1 million log records and checks whether the processing requirement can be satisfied.

```
#include <stdio.h>
#include <time.h>
int main() {
    long entries = 1000000;
    clock_t start, end;
    double time_taken;
    start = clock();
    long errors = 0;
    long warnings = 0;
    long normal = 0;
    for (long i = 0; i < entries; i++) {
        if (i % 20 == 0)
            errors++;
        else if (i % 10 == 0)
            warnings++;
        else
            normal++;
    }
```

```

}
end = clock();
time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Log Processing Results\n");
printf("-----\n");
printf("Total entries : %ld\n", entries);
printf("Errors : %ld\n", errors);
printf("Warnings : %ld\n", warnings);
printf("Normal : %ld\n", normal);
printf("Processing time: %.4f seconds\n", time_taken);
if (time_taken <= 120)
printf("Constraint satisfied: Processing time <= 2 minutes\n");
else
printf("Constraint violated: Processing time > 2 minutes\n");
printf("\nRecommended Linux commands:\n");
printf("awk, grep, sort, uniq, cut\n");
return 0;
}

```

OUTPUT :

```

Output

Log Processing Results
-----
Total entries : 1000000
Errors       : 50000
Warnings     : 50000
Normal       : 900000
Processing time: 0.0043 seconds
Constraint satisfied: Processing time <= 2 minutes

Recommended Linux commands:
awk, grep, sort, uniq, cut

=== Code Execution Successful ===

```

Conclusion: awk and grep are preferred because they process input efficiently without creating a separate process for every log entry. Avoiding shell loops and unnecessary commands helps the script meet the 2-minute constraint.

4. Faulty Process Consuming 90% CPU

Analysis

A background process is consuming 90% CPU. The system cannot be restarted because critical services must continue operating.

Constraints

- No system restart.
- Critical services must remain unaffected.
- Faulty process must be identified.
- CPU usage must be controlled.

Linux tools

1. Identify process

top

or:

ps aux --sort=-%cpu

2. Monitor process

top -p PID

3. Reduce priority

renice 10 -p PID

4. Stop process

If it is safe to terminate:

kill PID

If it does not respond:

kill -9 PID

The preferred approach is to first reduce its priority rather than immediately killing it.

C Program

```
#include <stdio.h>
```

```
int main() {
```

```
int pid;
```

```

float cpu;
printf("Enter faulty process PID: ");
scanf("%d", &pid);
printf("Enter CPU usage percentage: ");
scanf("%f", &cpu);
printf("\nProcess Management Analysis\n");
printf("-----\n");
printf("PID: %d\n", pid);
printf("CPU Usage: %.2f%%\n", cpu);
if (cpu >= 90) {
printf("Critical CPU usage detected.\n");
printf("Step 1: Monitor using top/ps.\n");
printf("Step 2: Reduce priority using renice.\n");
printf("Step 3: If necessary, terminate using kill.\n");
printf("Critical services remain unaffected.\n");
} else {
printf("CPU usage is within acceptable limits.\n");
}
return 0;
}

```

OUTPUT :

Output

```

Enter faulty process PID: 2456
Enter CPU usage percentage: 90

Process Management Analysis
-----
PID: 2456
CPU Usage: 90.00%
Critical CPU usage detected.
Step 1: Monitor using top/ps.
Step 2: Reduce priority using renice.
Step 3: If necessary, terminate using kill.
Critical services remain unaffected.

```

```

=== Code Execution Successful ===

```

Conclusion: The administrator should first identify and monitor the process using top/ps, lower its priority using renice, and terminate it only if necessary. This avoids a system restart and protects critical services.

5. RAM Allocation for 5 Virtual Machines

Analysis

Available physical RAM:

16 GB

Requirements:

- 5 VMs
- Each VM requires minimum 2 GB
- Host OS requires 4 GB

Minimum VM memory:

$5 \times 2 = 10 \text{GB}$

Host:

4GB

Total:

$10 + 4 = 14 \text{GB}$

Therefore:

$16 - 14 = 2 \text{GB}$

There are 2 GB remaining.

Optimized allocation

Resource	RAM
Host OS	4 GB
VM1	2 GB
VM2	2 GB
VM3	2 GB
VM4	2 GB
VM5	2 GB
Available for overhead/buffer	2 GB
Total	16 GB

The 2 GB remaining should not be assigned permanently to VMs. It should be retained as a safety margin for virtualization overhead and temporary host requirements.

Virtualization concepts

- Memory allocation: Give each VM its minimum 2 GB.
- Memory overcommit: Avoid excessive overcommit because all VMs require guaranteed minimum memory.
- Dynamic memory: If supported, allocate unused memory dynamically.
- Ballooning: Reclaim unused VM memory when the host requires it.
- Host reservation: Keep 4 GB reserved for the host.

C Program

```
#include <stdio.h>
int main() {
int total_ram = 16;
int host_ram = 4;
int vm_count = 5;
int vm_min = 2;
int vm_ram = vm_count * vm_min;
int used = host_ram + vm_ram;
int remaining = total_ram - used;
printf("Virtual Machine RAM Allocation\n");
printf("-----\n");
printf("Host OS : %d GB\n", host_ram);
for (int i = 1; i <= vm_count; i++) {
printf("VM%d : %d GB\n", i, vm_min);
}
printf("\nTotal RAM : %d GB\n", total_ram);
printf("RAM Allocated : %d GB\n", used);
printf("Remaining RAM : %d GB\n", remaining);
if (remaining >= 0) {
printf("\nConstraint satisfied.\n");
printf("All 5 VMs receive their minimum 2 GB.\n");
printf("Keep remaining RAM as virtualization overhead/buffer.\n");
} else {
```

```
printf("\nConstraint violated: Insufficient RAM.\n");
}  
return 0;  
}
```

OUTPUT :

```
Output  
  
Virtual Machine RAM Allocation  
-----  
Host OS : 4 GB  
VM1      : 2 GB  
VM2      : 2 GB  
VM3      : 2 GB  
VM4      : 2 GB  
VM5      : 2 GB  
  
Total RAM      : 16 GB  
RAM Allocated  : 14 GB  
Remaining RAM  : 2 GB  
  
Constraint satisfied.  
All 5 VMs receive their minimum 2 GB.  
Keep remaining RAM as virtualization overhead/buffer.
```

Conclusion: The minimum requirement is 14 GB, leaving 2 GB as a safety margin. Therefore, all five VMs can run smoothly without starving the host OS. Memory ballooning/dynamic allocation can further improve utilization when supported.